Principal Component Analysis
CS4330

Mathias Kölsch

with slides from Octavia I. Camps

Efficient Image Storage

Toy Example: Images with 3 pixels

Consider the following 3x1 images:

1	2	4	3	5	6
2	4	8	6	10	12
3	6	12	9	15	18

Storage requirement for 6 images? (1 pixel = 1 byte)

We need 18 = 3 x 6 bytes

Efficient Image Storage

Looking closer, we can see that all the images are very similar to each other: they are all the same image, scaled by a factor:

1		1	2		1	4		1
2	= 1 *	2	4	= 2 *	2	8	= 4 *	2
3		3	6		3	12		3
3		1	5		1	6		1
6	= 3 *	2	10	= 5 *	2	12	= 6 *	2
9		3	15		3	18		3

Efficient Image Storage

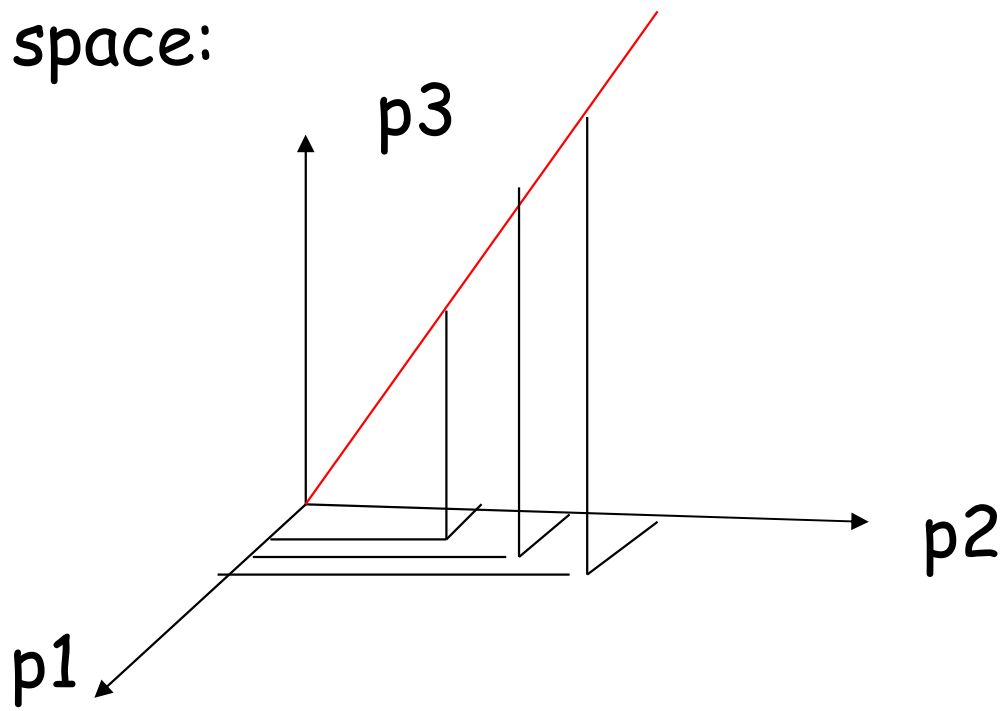
$$\begin{bmatrix}1\\2\\3\end{bmatrix} = 1 * \begin{bmatrix}1\\2\\3\end{bmatrix} \qquad \begin{bmatrix}2\\4\\6\end{bmatrix} = 2 * \begin{bmatrix}1\\2\\3\end{bmatrix} \qquad \begin{bmatrix}4\\8\\12\end{bmatrix} = 4 * \begin{bmatrix}1\\2\\3\end{bmatrix}$$

$$\begin{bmatrix}3\\6\\9\end{bmatrix} = 3 * \begin{bmatrix}1\\2\\3\end{bmatrix} \qquad \begin{bmatrix}5\\10\\15\end{bmatrix} = 5 * \begin{bmatrix}1\\2\\3\end{bmatrix} \qquad \begin{bmatrix}6\\12\\18\end{bmatrix} = 6 * \begin{bmatrix}1\\2\\3\end{bmatrix}$$

They can be stored using only 9 bytes (50% savings!):
Store one image (3 bytes) + the multiplying constants (6 bytes)

Geometrical Interpretation:

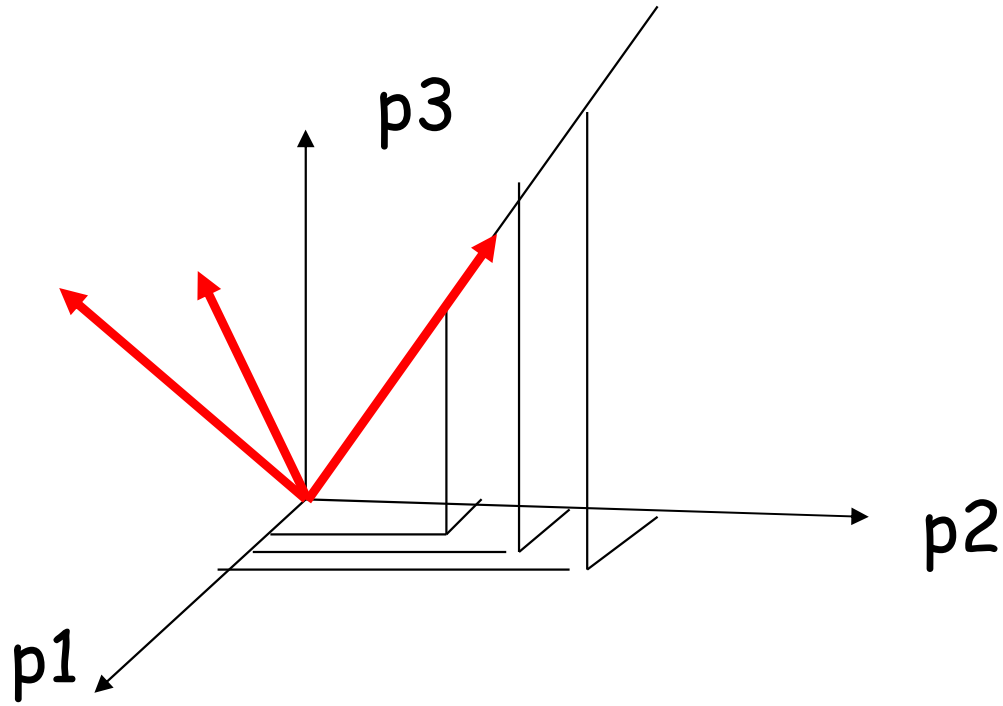
Consider each pixel in the image as a coordinate in a vector space. Then, each 3x1 template can be thought of as a point in a 3D space:



But in this example, all the points happen to belong to a line: a 1D subspace of the original 3D space.

Geometrical Interpretation:

Consider a new coordinate system where one of the axes is along the direction of the line:



In this coordinate system, every image has <u>only one</u> non-zero coordinate: we <u>*only*</u> need to store the direction of the line (a 3 bytes image) and the non-zero coordinate for each of the images (6 bytes).

Principal Component Analysis (PCA)

- Given a set of templates (features, descriptors, samples, …), how do we know if they can be compressed like in the previous example?
  - Look into the correlation between the templates
  - The tool for doing this is called PCA

PCA Theorem

Each N x N image can be represented as a N^2 x 1 (column) vector whose elements are the pixel values.

Let x_1 x_2 … x_n be a set of n N^2 x 1 vectors and let x be their average:

$$\mathbf{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{iN^2} \end{bmatrix} \qquad \bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^{i=n} \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{iN^2} \end{bmatrix}$$

PCA Theorem

Let X be the N^2 x n matrix with columns

$x_1 - \bar{x}, x_2 - \bar{x}, \ldots, x_n - \bar{x}$

that is, one row per feature vector element (= per pixel), and one column per image:

$$X = \left[\begin{array}{cccc} \mathbf{x}_1 - \bar{\mathbf{x}} & \mathbf{x}_2 - \bar{\mathbf{x}} & \cdots & \mathbf{x}_n - \bar{\mathbf{x}} \end{array} \right]$$

(Subtracting the mean is equivalent to translating the coordinate system to the location of the mean.)

PCA Theorem

Let $Q = X X^T$ be the $N^2 \times N^2$ matrix:

$$Q = XX^T = \begin{bmatrix} \mathbf{x}_1 - \bar{\mathbf{x}} & \mathbf{x}_2 - \bar{\mathbf{x}} & \cdots & \mathbf{x}_n - \bar{\mathbf{x}} \end{bmatrix} \begin{bmatrix} (\mathbf{x}_1 - \bar{\mathbf{x}})^T \\ (\mathbf{x}_2 - \bar{\mathbf{x}})^T \\ \vdots \\ (\mathbf{x}_n - \bar{\mathbf{x}})^T \end{bmatrix}$$

Notes:

1. Q is square
2. Q is symmetric
3. Q is the sample <u>*covariance*</u> matrix
4. Q can be very large (remember that N^2 is the number of pixels in the template)

PCA Theorem

Theorem:

Each x_j can be written as: $$\mathbf{x}_j = \bar{\mathbf{x}} + \sum_{i=1}^{i=n} g_{ji}\mathbf{e}_i$$

where e_i are the n eigenvectors of Q with non-zero eigenvalues. ($n=N^2$ actually, change in variable name…)

Notes:

1. The eigenvectors e_1 e_2 … e_n span an ***<u>eigenspace</u>***
2. e_1 e_2 … e_n are $N^2 \times 1$ orthonormal vectors (N x N-sized images).
3. The scalars g_{ji} are the coordinates of x_j in the space.
4. $$g_{ji} = (\mathbf{x}_j - \bar{\mathbf{x}}).\mathbf{e}_i$$

Using PCA to Compress Data

- Expressing x in terms of $e_1 \ldots e_n$ has not changed the size of the data
- However, if the templates are highly correlated many of the coordinates of x will be zero or close to zero.

Using PCA to Compress Data

- Sort the eigenvectors e_i according to their eigenvalue:

$$\lambda_1 \geq \lambda_2 \geq \ldots \lambda_n$$

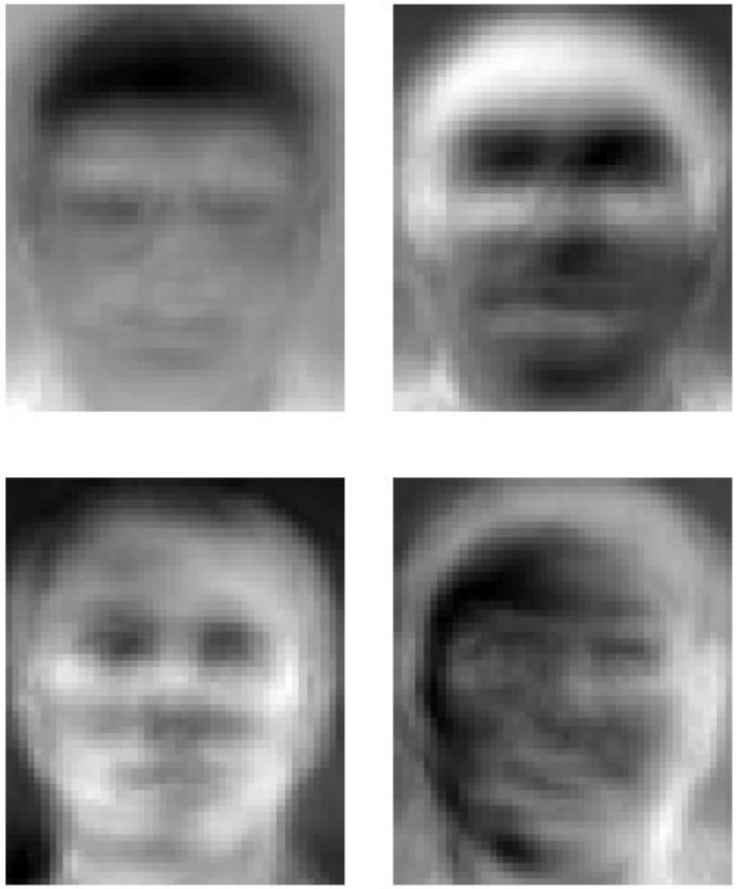
- Find k such that $\lambda_i \approx 0$ if $i > k$

- Then

$$\boxed{\mathbf{x}_j \approx \bar{\mathbf{x}} + \sum_{i=1}^{i=k} g_{ji}\mathbf{e}_i}$$

Example: Eigenfaces

- turn image patches into feature vectors
- calculate mean image
- apply PCA to find eigenvectors 1, 2, and 3



apply PCA to what?

Implementing PCA

- Need to find "first" k eigenvectors of Q:

$$Q = XX^T = \begin{bmatrix} \mathbf{x}_1 - \bar{\mathbf{x}} & \mathbf{x}_2 - \bar{\mathbf{x}} & \cdots & \mathbf{x}_n - \bar{\mathbf{x}} \end{bmatrix} \begin{bmatrix} (\mathbf{x}_1 - \bar{\mathbf{x}})^T \\ (\mathbf{x}_2 - \bar{\mathbf{x}})^T \\ \vdots \\ (\mathbf{x}_n - \bar{\mathbf{x}})^T \end{bmatrix}$$

Q is N^2 x N^2 where N^2 is the number of pixels in each image. For a 256 x 256 image, N^2 = 65536 !!

Finding ev of Q

$Q=XX^T$ is very large. Instead, consider the matrix $P=X^TX$

- Q and P are both symmetric, but $Q \neq P^T$
- Q is $N^2 \times N^2$, P is $n \times n$
- n is the number of training images, typically $n \ll N$

Finding ev of Q

Let e be an eigenvector of P with eigenvalue λ:

$$Pe = \lambda e$$
$$X^T X e = \lambda e$$
$$X X^T X e = \lambda X e$$
$$Q(Xe) = \lambda (Xe)$$

Xe is an eigenvector of Q also with eigenvalue λ!

Singular Value Decomposition (SVD)

Any m x n matrix X can be written as the product of 3 matrices:

$$X = UDV^T$$

Where:

- U is m x m and its columns are orthonormal vectors
- V is n x n and its columns are orthonormal vectors
- D is m x n diagonal and its diagonal elements are called the singular values of X, and are such that:

σ_1 , σ_2 , … σ_n , 0 and σ_1 = sqrt(λ_i)

SVD Properties

$$X = UDV^T$$

- The columns of U are the eigenvectors of XX^T
- The columns of V are the eigenvectors of X^TX
- The squares of the diagonal elements of D are the eigenvalues of XX^T and X^TX

Algorithm EIGENSPACE_LEARN

Assumptions:

1. Each image contains one object only.
2. Objects are imaged by a fixed camera .
3. Images are normalized in size N x N:
   - The image frame is the minimum rectangle enclosing the object.
4. Energy of pixels values is normalized to 1:
   - $\sum_i \sum_j I(i,j)^2 = 1$
5. The object is completely visible and unoccluded in all images.

Algorithm EIGENSPACE_LEARN

Getting the data:

For each object o to be represented, o = 1, …,O

1. Obtain n images
   1. n different photos of the same object o
   2. photos taken under different lighting conditions
   3. Possibly with other variation, s.a. smiling and not smiling
2. For each image p, p = 1, …, n:
   1. Segment o from the background
   2. Normalize the image size and energy
   3. Arrange the pixels as vectors $\mathbf{x}^o_p$

Algorithm EIGENSPACE_LEARN

Storing the data:

1. Find the average image vector $\bar{\mathbf{x}} = \frac{1}{n.o} \sum_{o=1}^{O} \sum_{p=1}^{n} x_p^o$

2. Assemble the matrix X:

$$X = \left[\begin{array}{cccc} \mathbf{x}_1^1 - \bar{\mathbf{x}} & \mathbf{x}_2^1 - \bar{\mathbf{x}} & \cdots & \mathbf{x}_n^o - \bar{\mathbf{x}} \end{array} \right]$$

3. Find the first k eigenvectors of XX^T: $e_1,\ldots,e_k$
   (use X^TX or SVD)

4. For each object o, each image p:
   - Compute the corresponding k-dimensional point:

$$\mathbf{g}_p^o = \left[\begin{array}{cccc} \mathbf{e}_1 & \mathbf{e}_2 & \cdots & \mathbf{e}_k \end{array} \right] (\mathbf{x}_p^o - \bar{\mathbf{x}})$$

Algorithm EIGENSPACE_IDENTIF

Recognizing an object from the DB:

1. Given an image, segment the object from the background

2. Normalize the size and energy, write it as a vector **i**

3. Compute the corresponding k-dimensional point:

$$\mathbf{g} = \begin{bmatrix} \mathbf{e}_1 & \mathbf{e}_2 & \cdots & \mathbf{e}_k \end{bmatrix} (\mathbf{i} - \bar{\mathbf{x}})$$

4. Find the closest $\boldsymbol{g}^{o}_{p}$ k-dimensional point to ***g***

Face Recognition

- Whose face is this? (perhaps in a mugshot)
- Issue:
  - What differences are important and what not?
  - Reduce the dimension of the images, while maintaining the "important" differences.
- One strategy:
  - Principal components analysis

- Many face detection resources at successor of Peter Kruizinga's page www.facedetection.com

Assume we have a set of n feature vectors $\boldsymbol{x}_i$ $(i = 1, \ldots, n)$ in $\mathbb{R}^d$. Write

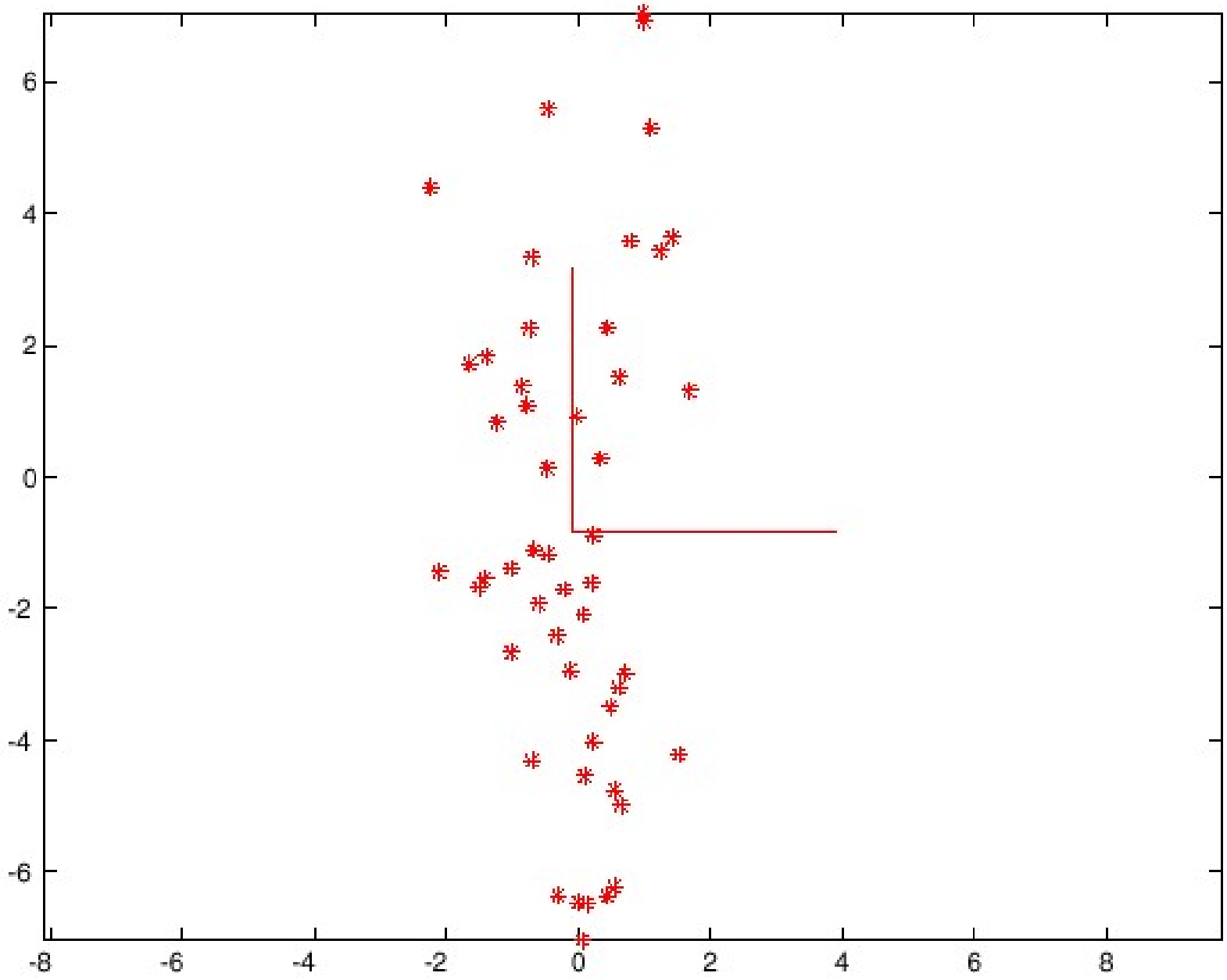
$$\boldsymbol{\mu} = \frac{1}{n} \sum_i \boldsymbol{x}_i$$

$$\Sigma = \frac{1}{n-1} \sum_i (\boldsymbol{x}_i - \boldsymbol{\mu})(\boldsymbol{x}_i - \boldsymbol{\mu})^T$$

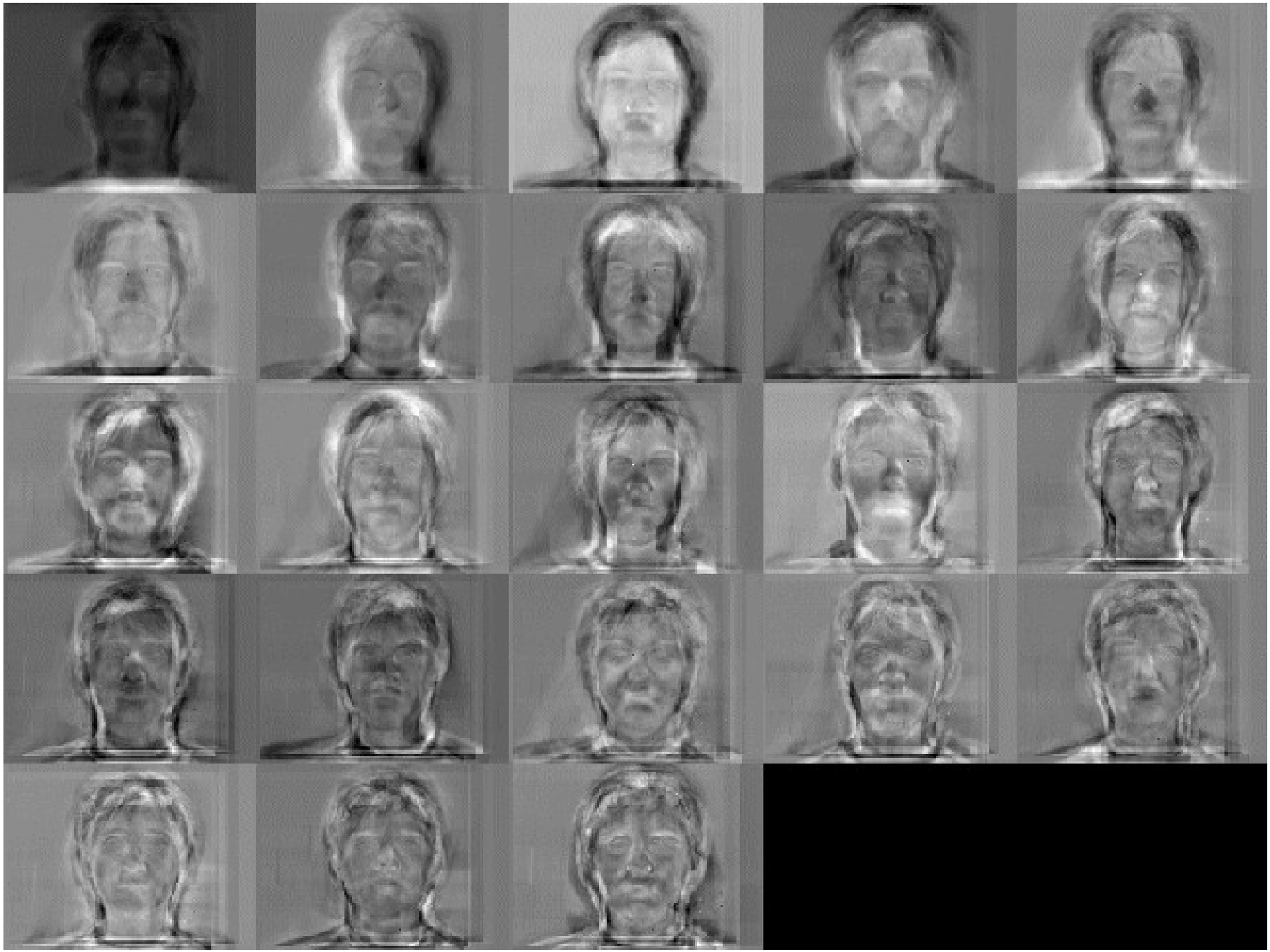
The unit eigenvectors of Σ — which we write as $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_d$, where the order is given by the size of the eigenvalue and $\boldsymbol{v}_1$ has the largest eigenvalue — give a set of features with the following properties:

- They are independent.
- Projection onto the basis $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ gives the k-dimensional set of linear features that preserves the most variance.

Algorithm 22.5: *Principal components analysis identifies a collection of linear features that are independent, and capture as much variance as possible from a dataset.*

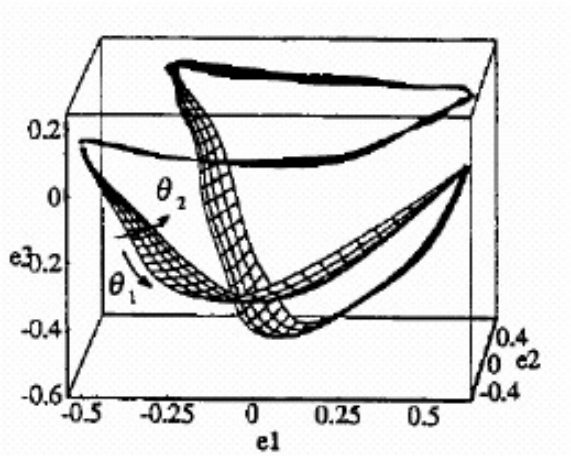

6
4
2
0
-2
-4
-6
-8
-6
-4
-2
0
2
4
6
8

Eigenfaces

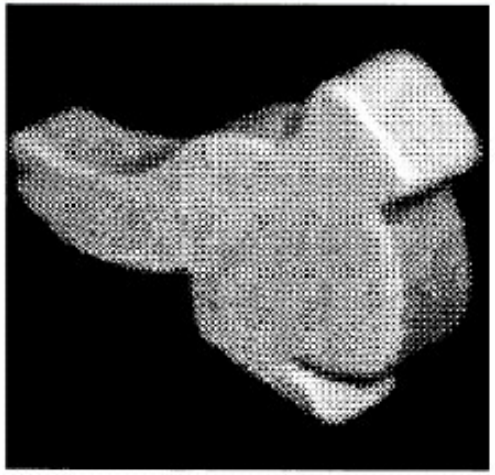


Appearance Manifolds

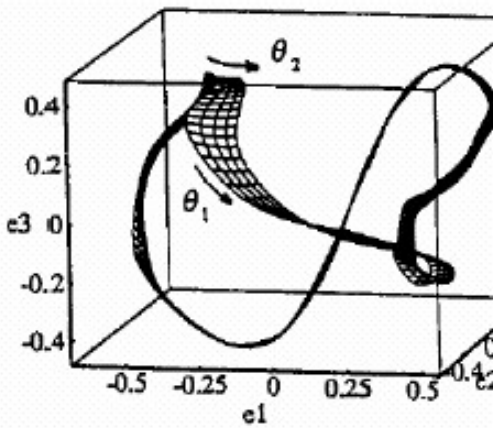
A



A

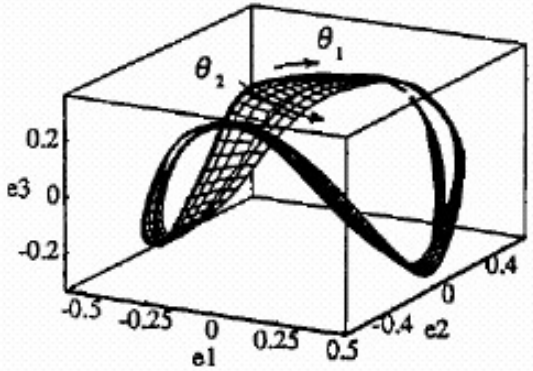


B

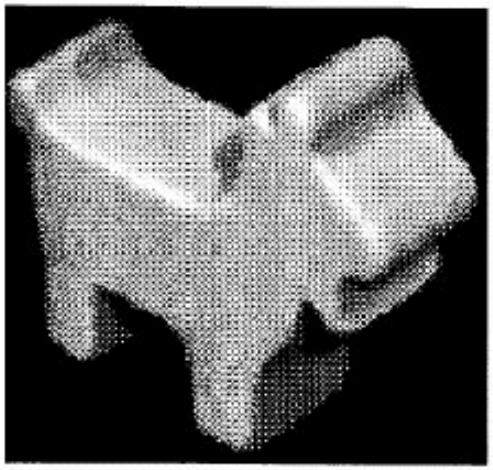


B

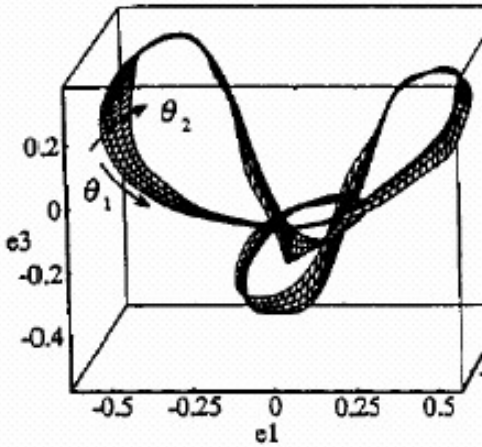
C



C



D



D

[Murase & Navar, 1996]